

The Confluent Flink Operator's Custom Resources

Four CRDs, what each one owns, and how they drive a Flink cluster

flink-stream reference environment

11 September 2026

Contents

1	What is actually installed	1
1.1	The control plane itself	2
2	FlinkDeployment	3
2.1	Two shapes, decided by one field	3
2.2	mode: native – the operator is not the only thing talking to Kubernetes	4
2.3	flinkConfiguration is a pass-through with additions	4
2.4	upgradeMode is the reason this CRD exists	5
2.5	The status is two independent state machines	5
2.6	The scale subresource, and why not to use it	6
2.7	What the operator does <i>not</i> do with this resource	7
3	FlinkSessionJob	7
4	FlinkStateSnapshot	7
4.1	The operator writes these resources too	8
4.2	Savepoint versus checkpoint, in this resource	9
5	FlinkBlueGreenDeployment	9
6	How the CRDs interface with Flink	10
6.1	High availability is the JobManager's job, not the operator's	10
6.2	What the operator's own RBAC reveals	11
6.3	The layering, end to end	11
7	Practical notes	11
7.1	Failure modes specific to these resources	12
8	Appendix: where to look	12

1 What is actually installed

Confluent's `flink-kubernetes-operator` Helm chart is Confluent's build of the Apache Flink Kubernetes Operator. It is the upstream project plus Confluent's packaging, support lifecycle and

one additional custom resource. The version mapping is not obvious from the chart version alone:

	Value in this environment
Helm chart	confluentinc/flink-kubernetes-operator 1.150.3
Operator image	confluentinc/cp-flink-kubernetes- operator:1.15.0-cp3
Flink runtime image	confluentinc/cp-flink, reporting 1.20.5-cp2
API group	flink.apache.org/v1beta1

Chart 1.150.3 is operator 1.15.0-cp3. The chart version is the operator version with the dots moved, and there is no way to infer one from the other without knowing that.

Installing the chart registers four cluster-wide CRDs, all namespaced, all on the single version v1beta1 – served and storage:

```
$ kubectl get crd -o name | grep flink.apache.org
customresourcedefinition.../flinkbluegreendeployments.flink.apache.org
customresourcedefinition.../flinkdeployments.flink.apache.org
customresourcedefinition.../flinksessionjobs.flink.apache.org
customresourcedefinition.../flinkstatesnapshots.flink.apache.org
```

Kind	Short name	Owns
FlinkDeployment	flinkdep	a Flink cluster, and optionally the one job running on it
FlinkSessionJob	sessionjob	a job submitted into an existing session cluster
FlinkStateSnapshot	flinksnp	one savepoint or checkpoint, as an object with a lifecycle
FlinkBlueGreenDeployment	flinkbgdep	two FlinkDeployments and a cutover between them

FlinkBlueGreenDeployment is the Confluent addition; the other three are upstream. Note the absence of anything Kafka-shaped – this operator has no KafkaTopic, Schema or Connector resource. Those belong to Confluent for Kubernetes (CFK), a separate operator with its own CRDs, which this environment does not use. Kafka here is a plain Helm-deployed StatefulSet.

1.1 The control plane itself

The operator deployment runs two containers, and both matter:

```
flink-kubernetes-operator-8c7578486-rgs45
  containers = flink-kubernetes-operator  flink-webhook
```

The webhook container serves admission over TLS, which is the reason cert-manager is a prerequisite. The MutatingWebhookConfiguration carries a cert-manager.io/inject-ca-from annota-

tion; with no cert-manager there is no certificate, the API server cannot call the webhook, and because both webhooks are registered failurePolicy: Fail, **every create is rejected**:

Webhook	Operations	Applies to
mutationwebhook.flink.apache.org	CREATE, UPDATE	flinkdeployments, flinksessionjobs
validationwebhook.flink.apache.org	CREATE, UPDATE	flinkdeployments, flinksessionjobs, flinkstatesnapshots

Running without cert-manager is supported – `--set webhook.create=false` – and the cost is precise rather than vague: defaulting and validation stop happening at admission time, so a malformed spec is accepted by `kubectl apply` and fails later in the operator log instead.

2 FlinkDeployment

This is the resource that does the work. Everything else in the group is defined in terms of it.

2.1 Two shapes, decided by one field

A `FlinkDeployment` describes a Flink cluster. Whether it also describes a *job* depends entirely on whether `spec.job` is present:

```
spec:
  image: flink-stream-jobs:1.0.0
  flinkVersion: v1_20
  serviceAccount: flink
  mode: native
  flinkConfiguration: { ... }
  jobManager: { resource: { memory: 1024m, cpu: 0.4 } }
  taskManager: { resource: { memory: 1600m, cpu: 0.8 } }
  job:                                # present -> application mode
    jarURI: local:///opt/flink/usrlib/flink-stream-jobs.jar
    entryClass: io.flinkstream.datastream.TransactionPipelineJob
    parallelism: 2
    upgradeMode: savepoint
    state: running
```

Delete the job block and the same resource becomes a session cluster: it starts empty and waits for submissions. In this environment `flinkdeployment-datastream.yaml` and `flinkdeployment-sql.yaml` are application clusters, and `flinkdeployment-session.yaml` is the same CRD with `spec.job` omitted and a `taskManager.replicas` count instead.

That single structural difference carries the whole application-versus-session trade: whether `main()` runs on the `JobManager` or on a client, whether an OOM is contained to one job, and whether resources can be accounted per job at all.

2.2 mode: native – the operator is not the only thing talking to Kubernetes

`spec.mode` selects how TaskManagers come into existence, and it is the field most often left at a default without understanding what it chose.

	native	standalone
Who creates TaskManager pods	the JobManager , via the Kubernetes API	the operator
Pod count	derived from parallelism and slots	fixed by <code>taskManager.replicas</code>
Rescaling	JobManager requests or releases pods	operator edits the replica count
Idle TaskManagers	released after <code>resourcemanager.taskmanager-timeout</code>	kept

In native mode the JobManager is itself a Kubernetes API client. This is why the chart ships a ServiceAccount, Role and RoleBinding for the *job*, separate from the operator's own RBAC:

rules:

```
- apiGroups: [""]
  resources: ["pods", "services", "configmaps"]
  verbs: ["create", "get", "list", "watch",
         "update", "patch", "delete", "deletecollection"]
```

The JobManager needs pods to create TaskManagers and configmaps for Kubernetes HA. A FlinkDeployment that references a ServiceAccount without these permissions starts a JobManager that comes up healthy and then never acquires a single TaskManager – a failure that looks like a scheduling problem and is an RBAC problem.

2.3 flinkConfiguration is a pass-through with additions

Most keys under `flinkConfiguration` are ordinary Flink configuration, handed to the cluster unchanged:

```
execution.checkpointing.interval: "30s"
state.backend.type: "rocksdb"
state.backend.incremental: "true"
high-availability.type: "kubernetes"
```

Mixed in among them, and syntactically indistinguishable, are keys the **operator** consumes and the Flink cluster never sees. They all begin `kubernetes.operator.` or `job.autoscaler.`:

```
kubernetes.operator.periodic.savepoint.interval: "1h"
kubernetes.operator.savepoint.history.max.count: "5"
kubernetes.operator.job.restart.failed: "true"
job.autoscaler.enabled: "true"
job.autoscaler.scaling.enabled: "false"
```

This is a genuine wart in the CRD design. Two different consumers read one map, and nothing in the schema distinguishes them, so a typo in an operator key is silently ignored rather than rejected – it simply looks like an unrecognised Flink option.

`kubernetes.operator.job.restart.failed` deserves its own mention because it covers a gap nothing else does. Flink’s `restart-strategy` handles *job* failures. A job whose `main()` threw never became a job, so no restart strategy applies – and in application mode `main()` runs on the JobManager and connects to Kafka, Schema Registry and the sinks immediately. This key is what makes the operator restart that case.

2.4 upgradeMode is the reason this CRD exists

A Deployment updates by rolling replicas. For a stateful Flink job that strategy is not merely suboptimal, it is wrong: TaskManagers hold disjoint slices of keyed state and participate in a coordinated snapshot protocol, so replacing them one at a time produces repeated recovery, not a gradual roll-out.

`spec.job.upgradeMode` encodes the correct sequence instead:

Mode	What the operator does	Correct when
savepoint	stop-with-savepoint, apply the change, restore from it	the default; state must survive
last-state	restart from the last checkpoint with no clean stop	faster, and only safe if the job graph is unchanged
stateless	discard state entirely	the logic changed incompatibly

The operator decides an upgrade is needed by comparing the incoming spec against `status.reconciliationStatus.lastReconciledSpec` – the full serialized previous spec, stored as a string in the status. Any change to the resource body triggers the sequence, including changes that do not affect the running job.

2.5 The status is two independent state machines

Reading a `FlinkDeployment` means reading two answers to two different questions, which is why the CRD’s printer columns show both:

```
$ kubectl -n flink-stream get flinkdep
```

NAME	JOB STATUS	LIFECYCLE STATE
fs-flink-stream-datastream	RUNNING	STABLE
fs-flink-stream-sql	RUNNING	STABLE

Field	Answers
<code>status.lifecycleState</code>	what the operator is doing: DEPLOYED, STABLE, ROLLING_BACK, FAILED
<code>status.jobStatus.state</code>	what Flink is doing: RUNNING, RESTARTING, FAILED

Field	Answers
<code>status.jobManagerDeploymentStatus</code>	whether the JobManager pod is READY, DEPLOYING, ERROR

STABLE plus RUNNING is the healthy pair. DEPLOYED plus RUNNING means the operator has applied the spec but has not yet seen the job stay up long enough to call it stable – and that distinction is what the rollback machinery is built on. `lastStableSpec` is recorded separately from `lastReconciledSpec`; if a new spec never reaches stability, the operator has a known-good spec to return to.

The status also carries a live view of the cluster that is useful precisely because it requires no port forward and no Flink UI:

```
status.clusterInfo.flink-version = "1.20.5-cp2"
status.clusterInfo.state-size    = "120504399"      # ~115 MiB, live
status.clusterInfo.total-memory = "2751463424"
status.jobStatus.jobId          = "63e7f885cf05c44ab1cf008201332535"
status.jobStatus.jobName        = "transaction-pipeline (DataStream)"
status.taskManager.replicas     = 1
```

`state-size` in a `kubectl get -o json` is the cheapest state-growth alarm available. A regular join with no TTL shows up here as a number that only ever increases.

2.6 The scale subresource, and why not to use it

`FlinkDeployment` – alone among the four – declares a scale subresource:

```
"scale": {
  "specReplicasPath": ".spec.taskManager.replicas",
  "statusReplicasPath": ".status.taskManager.replicas",
  "labelSelectorPath": ".status.taskManager.labelSelector"
}
```

This makes `kubectl scale flinkdep/... --replicas=4` work, and it makes a `FlinkDeployment` a legal `scaleTargetRef` for a `HorizontalPodAutoscaler`. Both are real capabilities and both are usually the wrong tool.

`spec.taskManager.replicas` is only meaningful in standalone mode; in native mode the JobManager derives pod count from parallelism, so scaling replicas adds TaskManagers that the job has no slots to place work on. And even in standalone mode, more TaskManagers without more parallelism changes nothing – parallelism is what distributes work, and changing parallelism requires a savepoint-and-restore cycle that `kubectl scale` does not perform.

Pointing an HPA at a `FlinkDeployment` is the specific version of this mistake worth naming: it scales on CPU, which is the wrong signal, at job granularity, which is the wrong unit. The Flink autoscaler exists because the right signals are source lag and per-vertex busyness, and the right unit is the individual operator.

2.7 What the operator does *not* do with this resource

The reconciliation loop understands lifecycle, not computation. It cannot tell that a regular join's state is unbounded, that a temporal join is resolving to NULL because log compaction removed the versions it needed, or that two sinks share a transactional-id prefix. Those are properties of the query, and they are found in operator metrics and in the output, not in the CR status.

3 FlinkSessionJob

A `FlinkSessionJob` is a job with no cluster of its own. It names an existing session-mode `FlinkDeployment` and is submitted into it:

```
apiVersion: flink.apache.org/v1beta1
kind: FlinkSessionJob
metadata:
  name: hybrid-session-job
spec:
  deploymentName: fs-flink-stream-session  # a FlinkDeployment with no spec.job
  job:
    jarURI: local:///opt/flink/usrlib/flink-stream-jobs.jar
    entryClass: io.flinkstream.hybrid.HybridTableJob
    parallelism: 2
    upgradeMode: stateless
    state: running
```

The `spec.job` block is deliberately the same shape as a `FlinkDeployment`'s, including `upgradeMode` and `state`, so the job-level vocabulary is identical in both places. What is missing is everything about the cluster: no image, no `flinkVersion`, no resources, no `flinkConfiguration`. Those belong to the deployment it targets.

Two operational details follow from the indirection.

jarURI is resolved by the JobManager, not by the operator. `local://` works here only because the jar is baked into the image the session cluster is running. A session cluster more usually receives jars it was not built with, which is what `http(s)://` and `s3://` are for – and which means the JobManager needs network access and credentials for wherever the jar lives.

The blast radius is the cluster. Every `FlinkSessionJob` pointed at one deployment shares its TaskManagers. One job exhausting the heap takes down its neighbours, and per-job resource accounting is not possible. This is the correct tool for interactive SQL and for short-lived work; it is the wrong tool for a production pipeline, which is why every job in this chart's default deploy is a `FlinkDeployment` instead.

Note also that `FlinkSessionJob` has no `scale` subresource and no `clusterInfo` in its status – there is no cluster for it to describe.

4 FlinkStateSnapshot

This is the newest of the upstream CRDs and the one that changes an operational practice rather than a deployment shape. Before it, savepoints were fields inside `FlinkDeployment` – a trigger

nonce in the spec and a history list in the status. Now each snapshot is its own object with its own lifecycle.

```
apiVersion: flink.apache.org/v1beta1
kind: FlinkStateSnapshot
metadata:
  name: manual-savepoint
spec:
  backoffLimit: 1
  jobReference:
    kind: FlinkDeployment
    name: fs-flink-stream-datastream
  savepoint:
    alreadyExists: false      # false = take a new one; true = register a path
    formatType: CANONICAL    # portable; NATIVE is faster, not portable
    disposeOnDelete: true    # delete the files when this object is deleted
```

jobReference.kind accepts FlinkDeployment or FlinkSessionJob, so the same resource takes a snapshot of either. The status is a small job-like state machine, surfaced directly in the printer columns:

```
$ kubectl -n flink-stream get flinksnp
```

NAME	PATH	SNAPSHOT STATE
...-savepoint-periodic-1789147026365	file:/flink-data/.../sp-3a5af9	COMPLETED
...-savepoint-periodic-1789150634498	file:/flink-data/.../sp-63e7f8	COMPLETED

with status.triggerId, status.triggerTimestamp, status.resultTimestamp, status.failures and status.error underneath. A savepoint becoming a first-class object means the trigger, the outcome and the resulting path are all observable by the same tooling that watches everything else in the cluster – and that a savepoint that failed leaves evidence rather than a gap in a history array.

4.1 The operator writes these resources too

The two snapshots above were not applied by anyone. They were created by the operator, because the DataStream deployment carries:

```
kubernetes.operator.periodic.savepoint.interval: "1h"
kubernetes.operator.savepoint.history.max.count: "5"
kubernetes.operator.savepoint.history.max.age: "24h"
```

This is the most interesting interface detail in the whole group: a controller reading one CRD emits instances of another, so the operator's periodic behaviour is expressed in exactly the same objects a human would write by hand. There is no hidden internal savepoint mechanism to learn separately.

Two asymmetries between operator-created and hand-written snapshots are worth knowing, both visible in the live objects:

- **Retention is by policy, not by ownership.** The periodic snapshots carry no ownerReferences. They are not garbage-collected when the deployment is deleted; they are pruned by the operator's own history.max.count and history.max.age. Delete the FlinkDeployment and its snapshot objects remain.

- **disposeOnDelete differs.** Operator-created periodic snapshots set `disposeOnDelete: false`, so the files survive the object. The hand-written example in `k8s-examples/` sets `true`, which is the right default for a deliberate pre-upgrade snapshot and the wrong one for an archive.

4.2 Savepoint versus checkpoint, in this resource

The CRD can represent either – there is a checkpoint block alongside savepoint – but they are not interchangeable, and the distinction is the reason to take a savepoint before a risky change:

	Checkpoint	Savepoint
Owner	the running job	you
Format	backend-specific, incremental	canonical, self-contained
Restore at a different parallelism	no	yes
Restore into a modified job graph	no	yes , if operator UIDs match

The “if operator UIDs match” clause is doing a great deal of work, and it is why every stateful operator in `TransactionPipelineJob` sets an explicit `.uid()`.

5 FlinkBlueGreenDeployment

The Confluent-specific resource, and structurally different from the other three: it is a **wrapper**, not a description of a cluster.

```
spec.template      # a FlinkDeployment (metadata + spec) to instantiate
spec.configuration # blue/green behaviour: cutover and abort policy
spec.ingress       # the endpoint that follows the active colour
```

`spec.template` holds an entire `FlinkDeployment` the way a `Deployment`’s `spec.template` holds a `Pod`. The controller instantiates it twice, in sequence, and manages the transition – which the status tracks with fields the other CRDs do not have:

```
status.blueGreenState      # active colour, and where in the transition
status.deploymentReadyTimestamp # when the new deployment became ready
status.savepointTriggerId  # the savepoint handing state from old to new
status.abortTimestamp      # when a failed transition was given up on
```

The mechanism is the one the `upgradeMode: savepoint` path already uses – stop-with-savepoint, start from it – with the difference that the new job is brought up and confirmed healthy *before* the old one is retired, rather than after. `savepointTriggerId` in the status is the handoff point made observable.

It is not exercised in this environment for a concrete reason: two full deployments run concurrently during the transition, which means twice the `TaskManager` memory. On a laptop-sized cluster that does not fit. On a real cluster it is the resource to reach for when a savepoint-restore gap of tens of seconds is not acceptable, and the price is understood to be double capacity during every upgrade.

6 How the CRDs interface with Flink

The layer boundaries are worth stating explicitly, because “the operator runs Flink” hides three different mechanisms.

Boundary	Mechanism
You to the API server	<code>kubectl apply</code> , then admission webhooks mutate and validate
Operator to the API server	watch the four CRDs plus pods, services, configmaps, secrets, deployments, ingresses, leases
Operator to Flink	the JobManager’s REST API – submit, stop-with-savepoint, trigger snapshot, poll status
JobManager to the API server	in native mode: create TaskManager pods; always: HA ConfigMaps
Flink to durable storage	checkpoints and savepoints, via <code>state.checkpoints.dir</code> – the operator never handles state bytes

The third row is the important one. The operator does not embed Flink; it is a REST client with a reconciliation loop. Everything it does to a running job – triggering a savepoint, cancelling, reading job status – is an HTTP call to the JobManager, which is why a JobManager that is up but unreachable produces an operator stuck reporting stale status rather than an operator that knows something is wrong.

The last row is the one people are surprised by. **State never passes through the operator.** A `FlinkStateSnapshot` contains a *path*; the bytes are written by TaskManagers directly to whatever `state.checkpoints.dir` points at. Deleting the operator, the CRs, or the entire namespace does not touch them.

6.1 High availability is the JobManager’s job, not the operator’s

With `high-availability.type: kubernetes`, the JobManager – not the operator – maintains its own leader election and metadata in a ConfigMap it creates:

```
$ kubectl -n flink-stream get cm fs-flink-stream-datastream-cluster-config-map
jobGraph-63e7f885cf05c44ab1cf008201332535 = r00ABXNyAGNvcmcuYXBhY2hlLmZs...
org.apache.flink.k8s.leader.dispatcher      = 30eb5820-...,pekko.tcp://...
org.apache.flink.k8s.leader.job-63e7f88... = 30eb5820-...,pekko.tcp://...
org.apache.flink.k8s.leader.resourcemanager= 30eb5820-...,pekko.tcp://...
org.apache.flink.k8s.leader.restserver      = 30eb5820-...,http://...:8081
```

Four leader-election entries and a serialized job graph pointer. The ConfigMap holds *pointers and leadership*, not state – the actual state is in the checkpoint directory. This is why the RBAC granted to the job’s ServiceAccount includes configmaps, and why deleting these ConfigMaps by hand while a job is running is a destructive act rather than a cleanup.

6.2 What the operator's own RBAC reveals

The ClusterRole is a precise statement of the operator's reach:

```

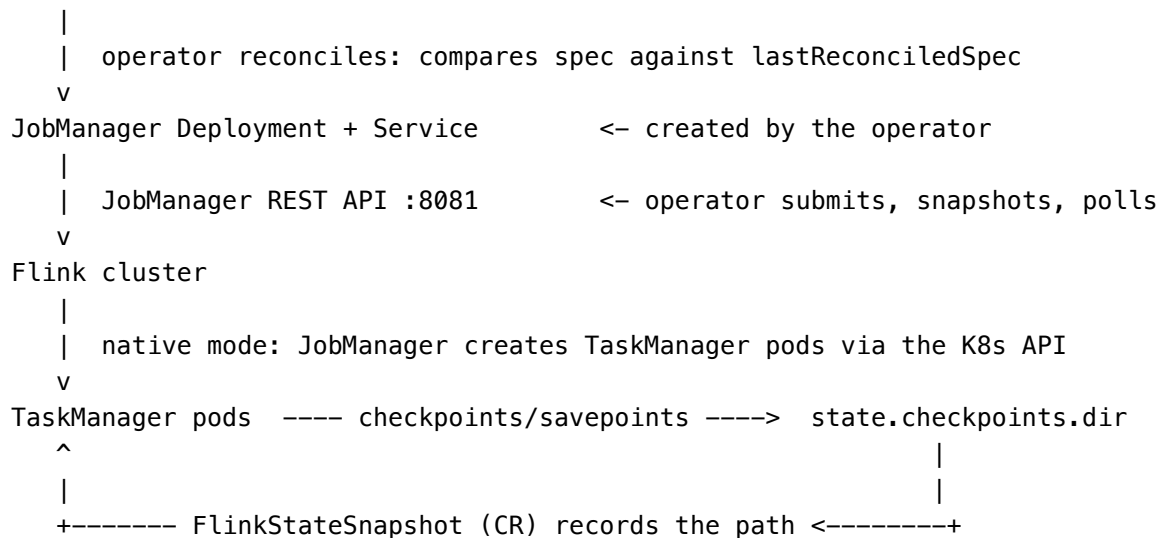
flink.apache.org    flinkdeployments, flinksessionjobs,
                    flinkstatesnapshots, flinkbluegreendeployments
                    (+ /finalizers)      get list watch create
                                         update patch delete
flink.apache.org    .../status          get update patch
""                 pods services events
                    configmaps secrets    full
apps                deployments, deployments/finalizers, replicasets
apps                deployments/scale      get update patch
networking.k8s.io   ingresses
coordination.k8s.io leases                full

```

Three things stand out. It holds **finalizers** on all four kinds, which is how deletion is made orderly – a FlinkDeployment delete triggers a job cancellation before the object disappears, so `kubectl delete` blocks until Flink has actually stopped. It has full access to **secrets**, because pod templates reference them – worth knowing when deciding whether the operator should be cluster-scoped, as it is here, or restricted with `watchNamespaces`. And it manages **leases**, for its own leader election when running more than one replica.

6.3 The layering, end to end

FlinkDeployment (CR)



7 Practical notes

Situation	What to reach for
A long-running production pipeline	FlinkDeployment with <code>spec.job.mode: native</code> , <code>upgradeMode: savepoint</code>

Situation	What to reach for
Interactive SQL, or many short jobs	one session <code>FlinkDeployment</code> plus <code>FlinkSessionJobs</code>
A deliberate snapshot before a risky change	<code>FlinkStateSnapshot</code> , <code>formatType: CANONICAL</code> , <code>disposeOnDelete: false</code>
Zero-downtime upgrade, capacity available	<code>FlinkBlueGreenDeployment</code>
Changing parallelism	<code>edit spec.job.parallelism</code> ; never <code>kubectl scale</code>
Reacting to load automatically	the Flink autoscaler; never an HPA

7.1 Failure modes specific to these resources

Symptom	Cause
Every <code>FlinkDeployment</code> create is rejected	no cert-manager, so the <code>failurePolicy: Fail</code> webhooks cannot be called
JobManager READY, zero TaskManagers ever appear	in native mode, the job's ServiceAccount cannot create pods
An operator config key has no effect	<code>kubernetes.operator.*</code> typo in <code>flinkConfiguration</code> ; unknown keys are ignored
Job never restarts after a <code>main()</code> exception	<code>kubernetes.operator.job.restart.failed</code> not set
<code>kubectl delete flinkdep</code> hangs	expected – the finalizer is waiting for the job to cancel
Savepoint objects outlive their deployment	periodic snapshots have no <code>ownerReferences</code> ; pruning is by history policy
State empty after an upgrade	<code>upgradeMode: stateless</code> , or operator UIDs moved
<code>kubectl scale</code> appears to work, nothing changes	replicas are not parallelism, and are inert in native mode

8 Appendix: where to look

Thing	File
Operator and cert-manager installation	<code>scripts/bootstrap.sh</code>
Chart and operator version pinning	<code>scripts/lib.sh</code>
Application-mode <code>FlinkDeployment</code>	<code>charts/flink-stream/templates/flink/flinkdeployment-datastream.yaml</code>
Session-mode <code>FlinkDeployment</code>	<code>charts/flink-stream/templates/flink/flinkdeployment-session.yaml</code>

Thing	File
Shared flinkConfiguration and pod template	charts/flink-stream/templates/flink/_flink-helpers.tpl
RBAC for the job's ServiceAccount	charts/flink-stream/templates/flink/rbac.yaml
FlinkSessionJob example	k8s-examples/flinksessionjob.yaml
FlinkStateSnapshot example	k8s-examples/flinkstatesnapshot.yaml
Day-to-day operations against these resources	docs/08-operations.md
Why the operator is necessary at all	docs/pdf/flink-the-model-and-its-edges.pdf
What it does not help with	docs/pdf/aggregation-joins-and-the-operator.pdf